Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки Кафедра
обчислювальної техніки

ЗВІТ

з лабораторної роботи №2

« Перетворення ER-діаграми на схему PostgreSQL»

з дисципліни: Бази Даних

Виконали: студенти групи ІМ-54
Дрига О.О.
Голубков В.С.
Ретізник А.Є.
Перевірив: Хмельницький А.А.

Київ - 2026

Тема: Перетворення ER-моделі на реляційну схему бази даних за допомогою SQL.

Мета: Набути практичних навичок проєктування реляційної схеми баз даних на основі існуючої ER-діаграми. Створити таблиці за допомогою SQL-інструкцій DDL у СУБД PostgreSQL, визначити типи даних, первинні та зовнішні ключі, а також застосувати необхідні обмеження (UNIQUE, NOT NULL, CHECK, DEFAULT). Заповнити таблиці тестовими даними за допомогою DML-інструкцій (INSERT) та перевірити цілісність створеної схеми.

Структура бази даних та скрипти ініціалізації (DDL та DML)

База даних складається з 10 взаємопов'язаних таблиць, що реалізують збереження даних про клієнтів, видавництва, авторів, категорії, книги (з їхніми зв'язками "багато до багатьох" через проміжні таблиці), а також замовлення, позиції замовлень та відгуки

DDL :

```
CREATE TABLE Customer (

    CustomerID SERIAL PRIMARY KEY,

    FirstName VARCHAR(50) NOT NULL,

    LastName VARCHAR(50) NOT NULL,

    Email VARCHAR(100) UNIQUE NOT NULL,

    Phone VARCHAR(20),

    RegistrationDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    ShippingAddressDefault TEXT

);

CREATE TABLE Publisher (

    PublisherID SERIAL PRIMARY KEY,

    NamePublisher VARCHAR(100) NOT NULL,

    Country VARCHAR(50)

);

CREATE TABLE Author (

    AuthorID SERIAL PRIMARY KEY,

    FirstName VARCHAR(50) NOT NULL,
```

```
    LastName VARCHAR(50) NOT NULL,
    Country VARCHAR(50)
);
CREATE TABLE Category (
    CategoryID SERIAL PRIMARY KEY,
    NameCategory VARCHAR(50) NOT NULL
);
CREATE TABLE Book (
    BookID SERIAL PRIMARY KEY,
    PublisherID INTEGER REFERENCES Publisher(PublisherID),
    ISBN VARCHAR(20) UNIQUE NOT NULL,
    Title VARCHAR(200) NOT NULL,
    PublicationYear INTEGER,
    Price NUMERIC(10,2) NOT NULL,
    StockQuantity INTEGER NOT NULL CHECK (StockQuantity >= 0)
);
CREATE TABLE BookAuthor (
    BookID INTEGER REFERENCES Book(BookID),
    AuthorID INTEGER REFERENCES Author(AuthorID),
    PRIMARY KEY (BookID, AuthorID)
);
CREATE TABLE BookCategory (
    BookID INTEGER REFERENCES Book(BookID),
    CategoryID INTEGER REFERENCES Category(CategoryID),
    PRIMARY KEY (BookID, CategoryID)
```

```
);

CREATE TABLE orders (

    OrderID SERIAL PRIMARY KEY,

    CustomerID INTEGER REFERENCES Customer(CustomerID),

    OrderDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    Status VARCHAR(20) NOT NULL,

    ShippingAddress TEXT NOT NULL,

    TotalAmount NUMERIC(10,2) NOT NULL

);

CREATE TABLE OrderItem (

    OrderItemID SERIAL PRIMARY KEY,

    OrderID INTEGER REFERENCES orders(OrderID),

    BookID INTEGER REFERENCES Book(BookID),

    Quantity INTEGER NOT NULL CHECK (Quantity > 0),

    PriceAtPurchase NUMERIC(10,2) NOT NULL

);

CREATE TABLE Review (

    ReviewID SERIAL PRIMARY KEY,

    CustomerID INTEGER REFERENCES Customer(CustomerID),

    OrderItemID INTEGER UNIQUE REFERENCES OrderItem(OrderItemID),

    Rating INTEGER NOT NULL CHECK (Rating >= 1 AND Rating <= 5),

    Comment TEXT,

    ReviewDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP

);
```

DML:

```
INSERT INTO Customer (FirstName, LastName, Email, Phone,
ShippingAddressDefault) VALUES

('Іван', 'Гриценко', 'ivan.franko@example.com', '+380501112233',
'Київ, вул. Хрещатик 1, кв 10'),

('Катерина', 'Іваненко', 'lesya.ukrainka@example.com',
'+380671112233', 'Львів, пл. Ринок 2, кв 5'),

('Тарас', 'Цимбал', 'taras.shevchenko@example.com', '+380631112233',
'Канів, вул. Дніпровська 3, кв 1'),

('Данило', 'Вовчок', 'marko.vovchok@example.com', '+380991112233',
'Одеса, вул. Морська 4, кв 12');

INSERT INTO Publisher (NamePublisher, Country) VALUES

('А-БА-БА-ГА-ЛА-МА-ГА', 'Україна'),

('Видавництво Старого Лева', 'Україна'),

('Клуб Сімейного Дозвілля', 'Україна');

INSERT INTO Author (FirstName, LastName, Country) VALUES

('Сергій', 'Жадан', 'Україна'),

('Джордж', 'Орвелл', 'Велика Британія'),

('Стівен', 'Кінг', 'США'),

('Ліна', 'Костенко', 'Україна');



INSERT INTO Category (NameCategory) VALUES

('Сучасна українська література'),

('Антиутопія'),

('Жахи'),

('Поезія');

INSERT INTO Book (PublisherID, ISBN, Title, PublicationYear, Price,
StockQuantity) VALUES

(1, '978-617-585-186-3', 'Інтернат', 2017, 300.00, 50),
```

```
(2, '978-966-14-1234-5', '1984', 2021, 250.00, 120),

(3, '978-966-14-1234-6', 'Воно', 2019, 450.00, 15),

(1, '978-617-585-186-4', 'Триста поезій', 2020, 350.00, 30);

INSERT INTO orders (CustomerID, Status, ShippingAddress, TotalAmount)
VALUES

(1, 'Оплачено', 'Київ, вул. Хрещатик 1, кв 10', 550.00),

(2, 'Очікує обробки', 'Львів, пл. Ринок 2, кв 5', 450.00),

(3, 'Доставлено', 'Канів, вул. Дніпровська 3, кв 1', 350.00),

(4, 'Відправлено', 'Одеса, вул. Морська 4, кв 12', 300.00);

INSERT INTO BookAuthor (BookID, AuthorID) VALUES

(1, 1),

(2, 2),

(3, 3),

(4, 4);

INSERT INTO BookCategory (BookID, CategoryID) VALUES

(1, 1),

(2, 2),

(3, 3),

(4, 4);

INSERT INTO OrderItem (OrderID, BookID, Quantity, PriceAtPurchase)
VALUES

(1, 1, 1, 300.00),

(1, 2, 1, 250.00),

(2, 3, 1, 450.00),

(3, 4, 1, 350.00),

(4, 1, 1, 300.00);
```

```sql
INSERT INTO Review (CustomerID, OrderItemID, Rating, Comment) VALUES

(1, 1, 5, 'Дуже сильна книга, рекомендую!'),

(1, 2, 4, 'Класика, але трохи важко читається.'),

(2, 3, 5, 'Страшно цікаво!'),

(3, 4, 5, 'Неймовірні вірші.'),

(4, 5, 5, 'Швидка доставка, якість друку супер.');
```

TEST

```sql
SELECT table_name

FROM information_schema.tables

WHERE table_schema = 'public'

ORDER BY table_name;



SELECT 'customer' AS tbl, COUNT(*) AS rows_count FROM customer

UNION ALL SELECT 'publisher',    COUNT(*) FROM publisher

UNION ALL SELECT 'author',       COUNT(*) FROM author

UNION ALL SELECT 'category',     COUNT(*) FROM category

UNION ALL SELECT 'book',         COUNT(*) FROM book

UNION ALL SELECT 'bookauthor',   COUNT(*) FROM bookauthor

UNION ALL SELECT 'bookcategory', COUNT(*) FROM bookcategory

UNION ALL SELECT 'orders',       COUNT(*) FROM orders

UNION ALL SELECT 'orderitem',    COUNT(*) FROM orderitem

UNION ALL SELECT 'review',       COUNT(*) FROM review;
```

```sql
SELECT b.title, p.namepublisher,

       a.firstname || ' ' || a.lastname AS author,

       c.namecategory

FROM book b

JOIN publisher p    ON p.publisherid = b.publisherid

JOIN bookauthor ba  ON ba.bookid = b.bookid

JOIN author a       ON a.authorid = ba.authorid

JOIN bookcategory bc ON bc.bookid = b.bookid

JOIN category c     ON c.categoryid = bc.categoryid;



SELECT o.orderid, c.lastname, o.status, b.title,

       oi.quantity, oi.priceatpurchase

FROM orders o

JOIN customer c   ON c.customerid = o.customerid

JOIN orderitem oi ON oi.orderid = o.orderid

JOIN book b       ON b.bookid = oi.bookid

ORDER BY o.orderid;



SELECT c.lastname, b.title, r.rating, r.comment

FROM review r

JOIN customer c   ON c.customerid = r.customerid

JOIN orderitem oi ON oi.orderitemid = r.orderitemid

JOIN book b       ON b.bookid = oi.bookid;



SELECT o.orderid, o.totalamount,
```

```
       SUM(oi.quantity * oi.priceatpurchase) AS calc
FROM orders o
JOIN orderitem oi ON oi.orderid = o.orderid
GROUP BY o.orderid, o.totalamount
ORDER BY o.orderid;




BEGIN


    BEGIN
        INSERT INTO customer (firstname, lastname, email)
        VALUES ('Тест', 'Дубль', 'ivan.franko@example.com');
        RAISE NOTICE 'C1 ПРОВАЛ: дубль Email вставився';
    EXCEPTION WHEN OTHERS THEN
        RAISE NOTICE 'C1 OK (дубль Email): [%] %', SQLSTATE, SQLERRM;
    END;




    BEGIN
        INSERT INTO book (publisherid, isbn, title, price,
stockquantity)
        VALUES (1, '000-0-00-000000-0', 'Тест', 100, -5);
        RAISE NOTICE 'C2 ПРОВАЛ: від''ємний StockQuantity вставився';
    EXCEPTION WHEN OTHERS THEN
        RAISE NOTICE 'C2 OK (StockQuantity = -5): [%] %', SQLSTATE,
SQLERRM;
```

```
    END;




    BEGIN
        INSERT INTO review (customerid, orderitemid, rating) VALUES
(1, NULL, 6);
        RAISE NOTICE 'C3 ПРОВАЛ: Rating = 6 вставився';
    EXCEPTION WHEN OTHERS THEN
        RAISE NOTICE 'C3 OK (Rating = 6): [%] %', SQLSTATE, SQLERRM;
    END;




    BEGIN
        INSERT INTO review (customerid, orderitemid, rating) VALUES
(1, NULL, 0);
        RAISE NOTICE 'C4 ПРОВАЛ: Rating = 0 вставився';
    EXCEPTION WHEN OTHERS THEN
        RAISE NOTICE 'C4 OK (Rating = 0): [%] %', SQLSTATE, SQLERRM;
    END;




    BEGIN
        INSERT INTO orders (customerid, status, shippingaddress,
totalamount)
        VALUES (999, 'Тест', 'Київ', 100);
        RAISE NOTICE 'C5 ПРОВАЛ: замовлення неіснуючого клієнта
вставилось';
```

```
    EXCEPTION WHEN OTHERS THEN

        RAISE NOTICE 'C5 OK (customerid = 999): [%] %', SQLSTATE,
SQLERRM;

    END;




    BEGIN

        INSERT INTO review (customerid, orderitemid, rating) VALUES
(1, 1, 3);

        RAISE NOTICE 'C6 ПРОВАЛ: другий відгук на позицію вставився';

    EXCEPTION WHEN OTHERS THEN

        RAISE NOTICE 'C6 OK (другий відгук на orderitemid = 1): [%]
%', SQLSTATE, SQLERRM;

    END;




    BEGIN

        INSERT INTO orderitem (orderid, bookid, quantity,
priceatpurchase)

        VALUES (1, 1, 0, 300);

        RAISE NOTICE 'C7 ПРОВАЛ: Quantity = 0 вставився';

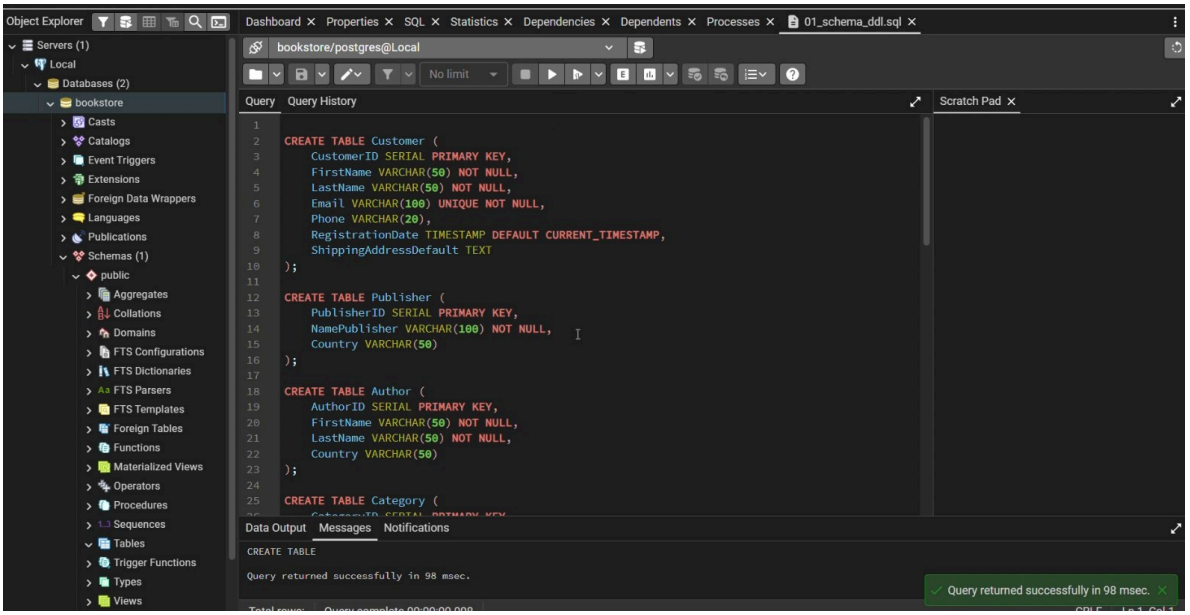
    EXCEPTION WHEN OTHERS THEN

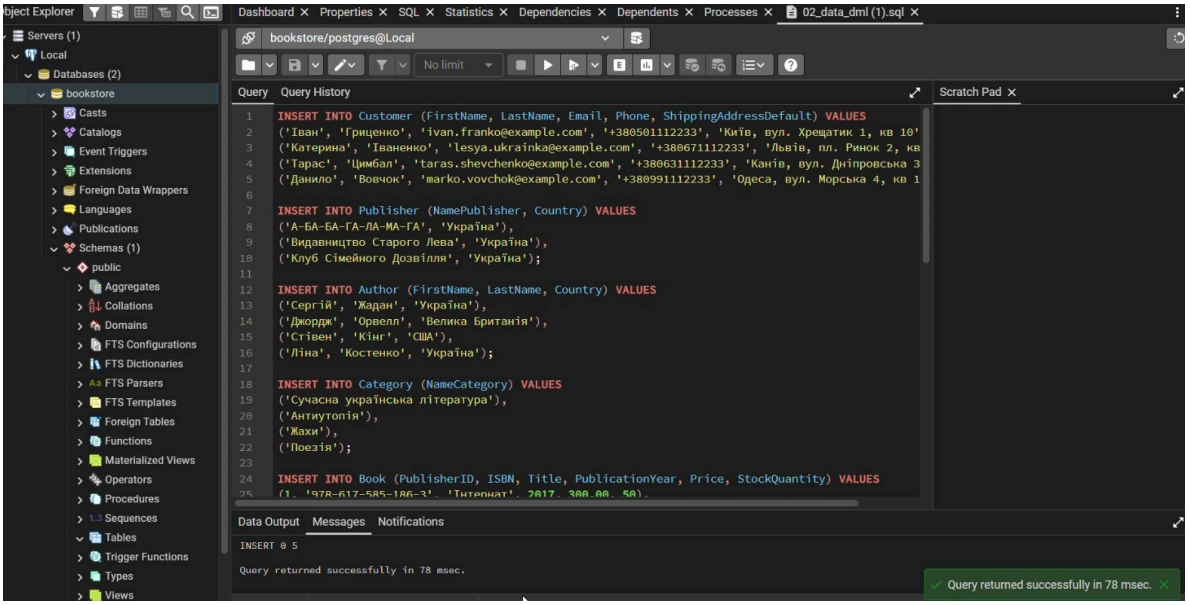
        RAISE NOTICE 'C7 OK (Quantity = 0): [%] %', SQLSTATE,
SQLERRM;

    END;

END
```

Успішне виконання DDL та DML запитів у pgAdmin:

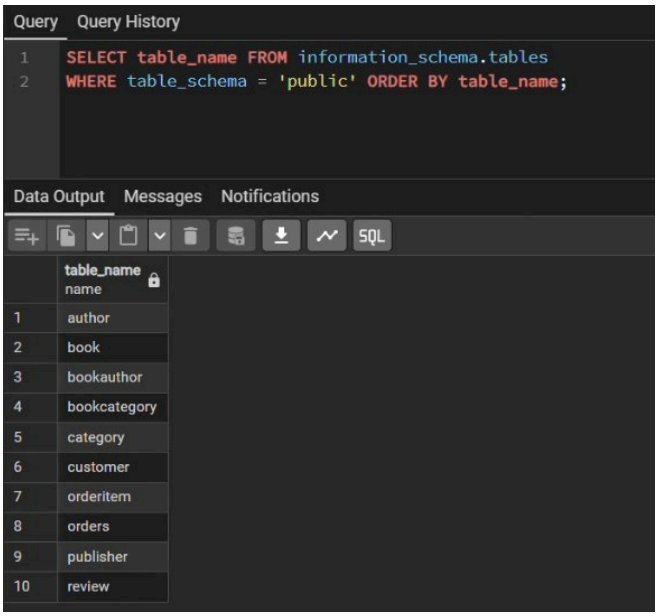


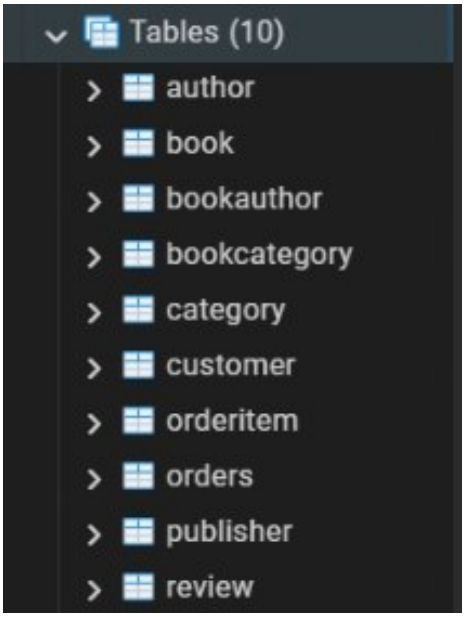


Перевірка структури та цілісності даних:

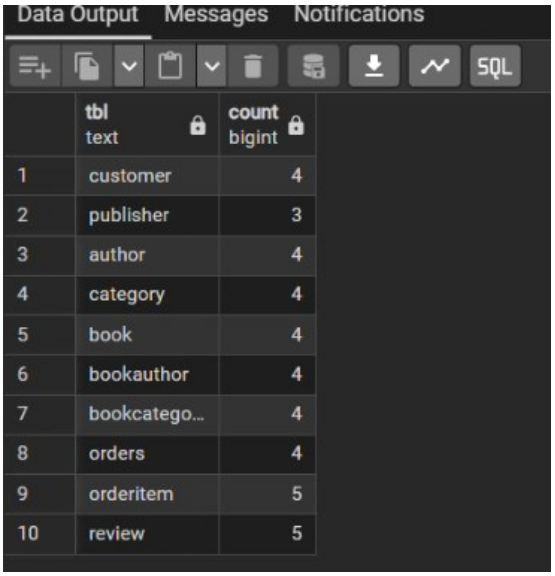
Після виконання скриптів було проведено верифікацію наповнення бази даних:

За допомогою запиту до information_schema.tables підтверджено наявність усіх 10 створених таблиць у схемі public. Перевірка наявності таблиць у базі даних:





Перевірка кількості рядків за допомогою конструкції UNION ALL підтвердила очікуваний обсяг даних: 4 клієнти, 3 видавництва, 4 автори, 4 категорії, 4 книги, 4 записи у зв'язках авторів та категорій, 4 замовлення, 5 позицій замовлень та 5 відгуків.



Аналітичні запити (JOIN)

Для перевірки реляційних зв'язків виконано вибірки даних із застосуванням JOIN:

Книги з деталями: Зведено інформацію про назви книг, їхніх видавців, авторів та категорії. Вибірка книг з інформацією про авторів, видавців та категорії:

```sql
SELECT b.title, p.namepublisher, a.firstname || ' ' || a.lastname AS author, c.namecategory
FROM book b
JOIN publisher p ON p.publisherid = b.publisherid
JOIN bookauthor ba ON ba.bookid = b.bookid
JOIN author a ON a.authorid = ba.authorid
JOIN bookcategory bc ON bc.bookid = b.bookid
JOIN category c ON c.categoryid = bc.categoryid;
```

Data Output | Messages | Notifications

Showing rows: 1 to 4

	title character varying (200)	namepublisher character varying (100)	author text	namecategory character varying (50)
1	Інтернат	А-БА-БА-ГА-ЛА-МА-ГА	Сергій Жадан	Сучасна українська літерату...
2	1984	Видавництво Старого Ле...	Джордж Ор...	Антиутопія
3	Воно	Клуб Сімейного Дозвілля	Стівен Кінг	Жахи
4	Триста поезій	А-БА-БА-ГА-ЛА-МА-ГА	Ліна Костен...	Поезія

Деталі замовлень: Виведено замовлення клієнтів із прив'язкою до придбаних позицій, кількості та ціни на момент покупки. Вибірка замовлень клієнтів із товарними позиціями:

```sql
SELECT o.orderid, c.lastname, o.status, b.title, oi.quantity, oi.priceatpurchase
FROM orders o
JOIN customer c ON c.customerid = o.customerid
JOIN orderitem oi ON oi.orderid = o.orderid
JOIN book b ON b.bookid = oi.bookid
ORDER BY o.orderid;
```

Data Output | Messages | Notifications

Showing rows: 1 to 5

	orderid integer	lastname character varying (50)	status character varying (20)	title character varying (200)	quantity integer	priceatpurchase numeric (10,2)
1	1	Гриценко	Оплачено	Інтернат	1	300.00
2	1	Гриценко	Оплачено	1984	1	250.00
3	2	Іваненко	Очікує обробки	Воно	1	450.00
4	3	Цимбал	Доставлено	Триста поезій	1	350.00
5	4	Вовчок	Відправлено	Інтернат	1	300.00

Відгуки: Отримано список відгуків з інформацією про клієнта, книгу, оцінку та текст коментаря.

Валідація сум: Виконано запит, який згрупував замовлення та математично підтвердив, що загальна сума замовлення (TotalAmount) коректно збігається із сумою вартості окремих позицій (SUM(Quantity * PriceAtPurchase)).

Вибірка відгуків та математична перевірка суми чека :

```sql
SELECT o.orderid, c.lastname, o.status, b.title, oi.quantity, oi.priceatpurchase
FROM orders o
JOIN customer c ON c.customerid = o.customerid
JOIN orderitem oi ON oi.orderid = o.orderid
JOIN book b ON b.bookid = oi.bookid
ORDER BY o.orderid;
```

Data Output | Messages | Notifications

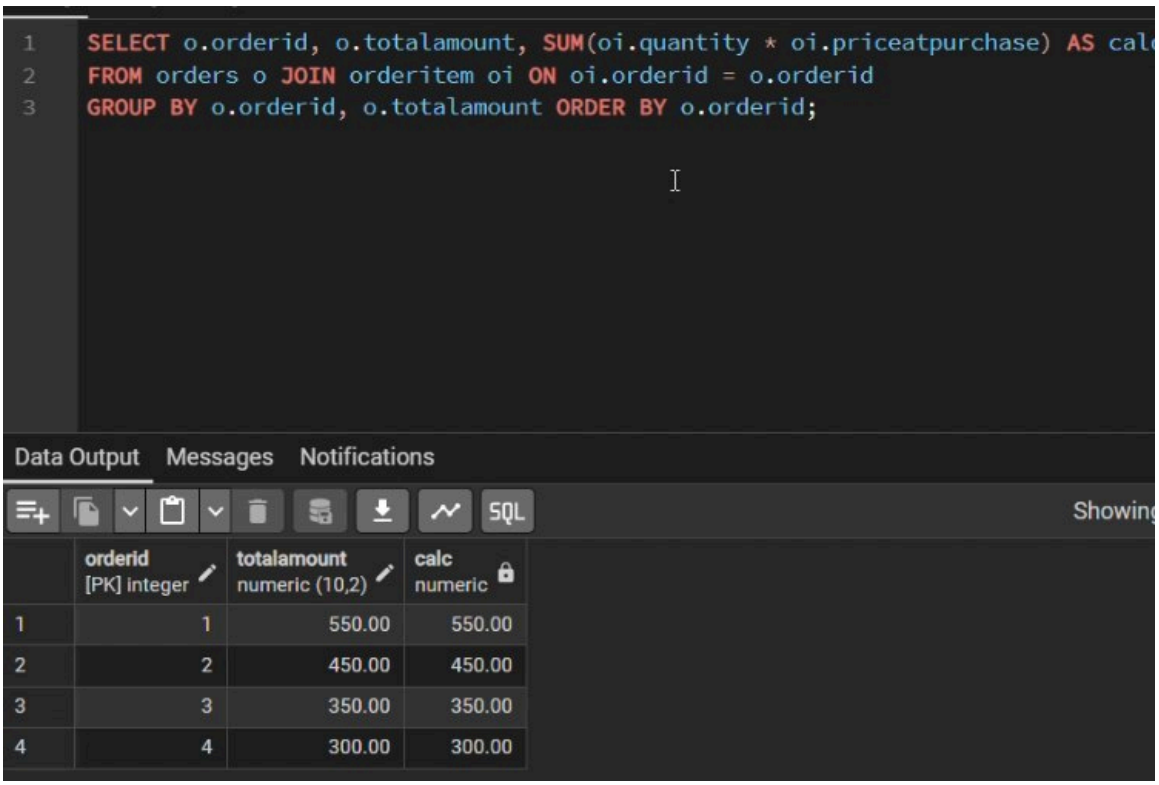
Showing rows: 1 to 5

	orderid integer	lastname character varying (50)	status character varying (20)	title character varying (200)	quantity integer	priceatpurchase numeric (10,2)
1	1	Гриценко	Оплачено	Інтернат	1	300.00
2	1	Гриценко	Оплачено	1984	1	250.00
3	2	Іваненко	Очікує обробки	Воно	1	450.00
4	3	Цимбал	Доставлено	Триста поезій	1	350.00
5	4	Вовчок	Відправлено	Інтернат	1	300.00

```sql
SELECT c.lastname, b.title, r.rating, r.comment
FROM review r
JOIN customer c ON c.customerid = r.customerid
JOIN orderitem oi ON oi.orderitemid = r.orderitemid
JOIN book b ON b.bookid = oi.bookid;
```

Data Output | Messages | Notifications

	lastname character varying (50)	title character varying (200)	rating integer	comment text
1	Гриценко	Інтернат	5	Дуже сильна книга, рекомендую!
2	Гриценко	1984	4	Класика, але трохи важко читаєтьс...
3	Іваненко	Воно	5	Страшно цікаво!
4	Цимбал	Триста поезій	5	Неймовірні вірші.
5	Вовчок	Інтернат	5	Швидка доставка, якість друку суп...

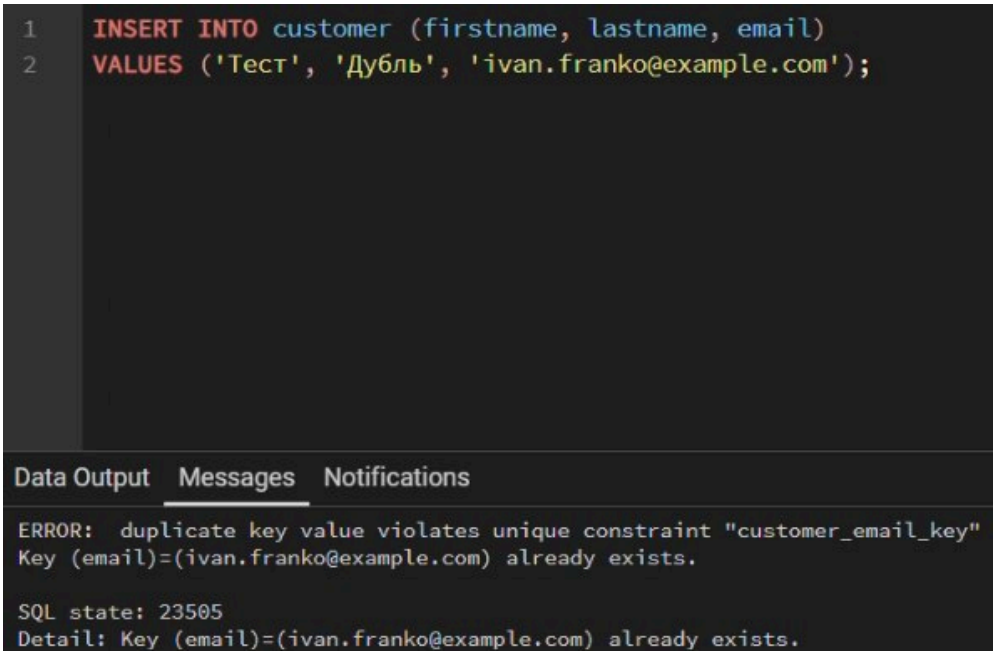


Негативне тестування обмежень (Constraints)

Для гарантування цілісності бази даних успішно проведено ряд негативних тестів, які підтвердили спрацювання обмежень:

- Унікальність Email (UNIQUE): Спроба додати користувача з існуючим email (ivan.franko@example.com) викликала помилку customer_email_key.
- Від'ємний залишок (CHECK): Спроба вказати кількість книг на складі -5 заблокована обмеженням book_stockquantity_check.

Спрацювання обмежень UNIQUE для email та CHECK для залишку на складі :



```
1  INSERT INTO book (publisherid, isbn, title, price, stockquantity)
2  VALUES (1, '000-0-00-000000-0', 'Тест', 100, -5);
```

Data Output Messages Notifications

```
ERROR:  new row for relation "book" violates check constraint "book_stockquantity_check"
Failing row contains (5, 1, 000-0-00-000000-0, Тест, null, 100.00, -5).

SQL state: 23514
Detail: Failing row contains (5, 1, 000-0-00-000000-0, Тест, null, 100.00, -5).
```

Некоректний рейтинг (CHECK): Вставка відгуків з оцінкою 6 та 0 відхилена через спрацювання review_rating_check.

Спрацювання обмеження CHECK для рейтингу відгуків:

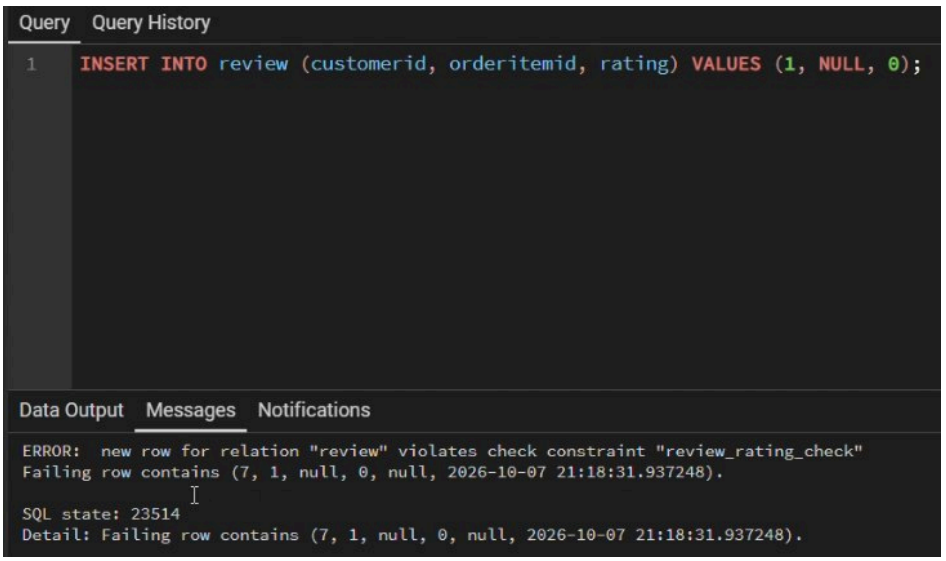
Query Query History

```
1  INSERT INTO review (customerid, orderitemid, rating) VALUES (1, NULL, 6);
```

Data Output Messages Notifications

```
ERROR:  new row for relation "review" violates check constraint "review_rating_check"
Failing row contains (6, 1, null, 6, null, 2026-10-07 21:18:08.469143).

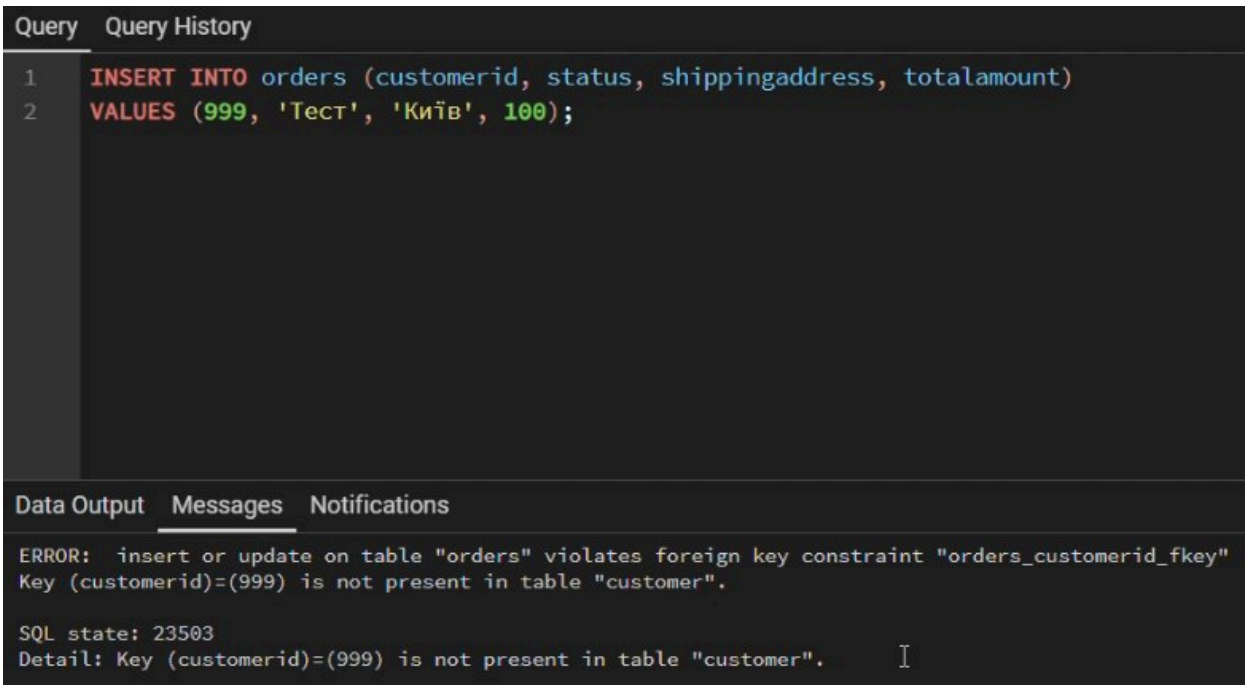
SQL state: 23514
Detail: Failing row contains (6, 1, null, 6, null, 2026-10-07 21:18:08.469143).
```

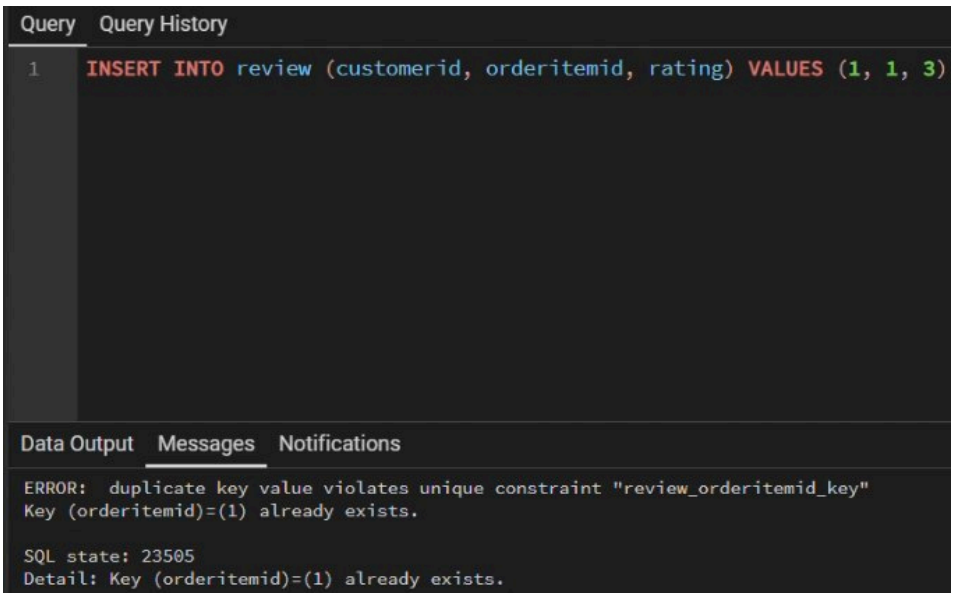


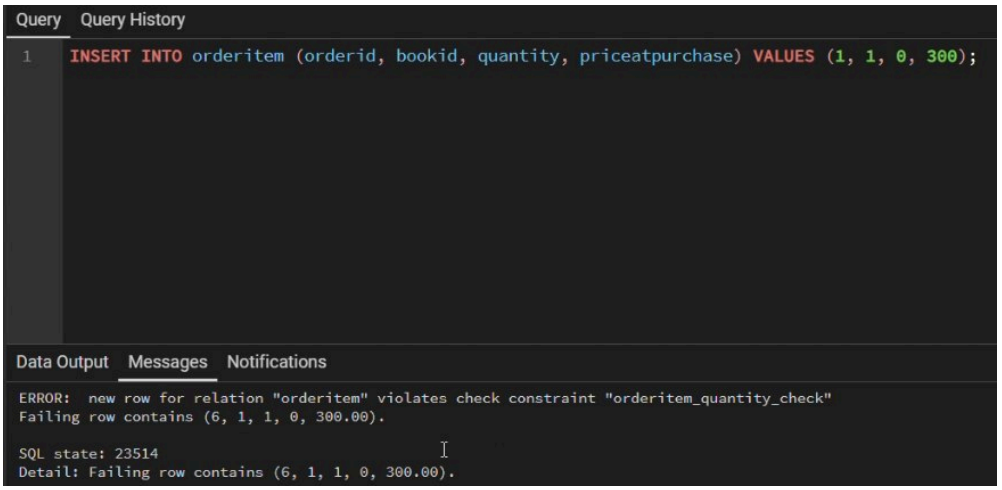
Порушення зовнішнього ключа (FOREIGN KEY): Спроба створити замовлення для неіснуючого клієнта (ID=999) видала помилку orders_customerid_fkey.

Дублювання відгуків (UNIQUE): Спроба залишити другий відгук на ту саму позицію замовлення (ID=1) була заблокована review_orderitemid_key.

Порушення цілісності зовнішнього ключа та дублювання відгуку:







Нульова кількість у замовленні (CHECK): Додавання позиції замовлення з Quantity = 0 відхилено обмеженням orderitem_quantity_check.

Фінальна перевірка: Повторний підрахунок рядків підтвердив, що жоден з некоректних записів не потрапив до бази даних.

Спрацювання обмеження на кількість та фінальна перевірка цілісності:

Query | Query History

```sql
SELECT 'customer' AS tbl, COUNT(*) FROM customer
UNION ALL SELECT 'publisher', COUNT(*) FROM publisher
UNION ALL SELECT 'author', COUNT(*) FROM author
UNION ALL SELECT 'category', COUNT(*) FROM category
UNION ALL SELECT 'book', COUNT(*) FROM book
UNION ALL SELECT 'bookauthor', COUNT(*) FROM bookauthor
UNION ALL SELECT 'bookcategory', COUNT(*) FROM bookcategory
UNION ALL SELECT 'orders', COUNT(*) FROM orders
UNION ALL SELECT 'orderitem', COUNT(*) FROM orderitem
UNION ALL SELECT 'review', COUNT(*) FROM review;
```

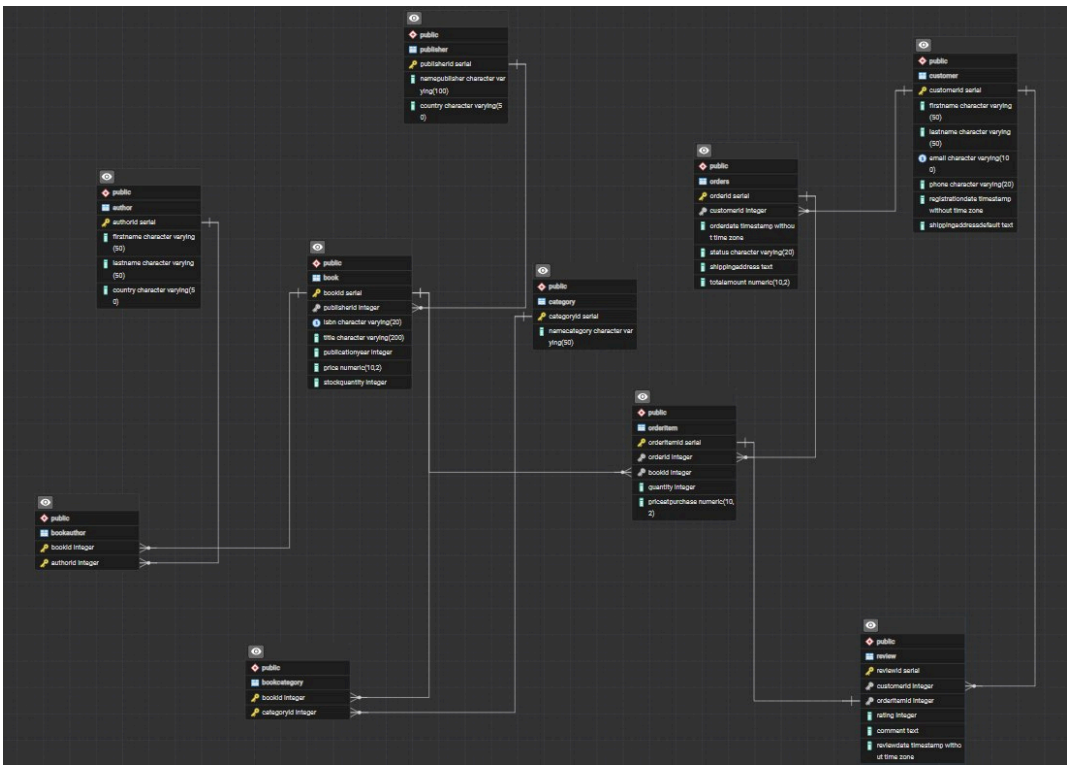
Data Output | Messages | Notifications

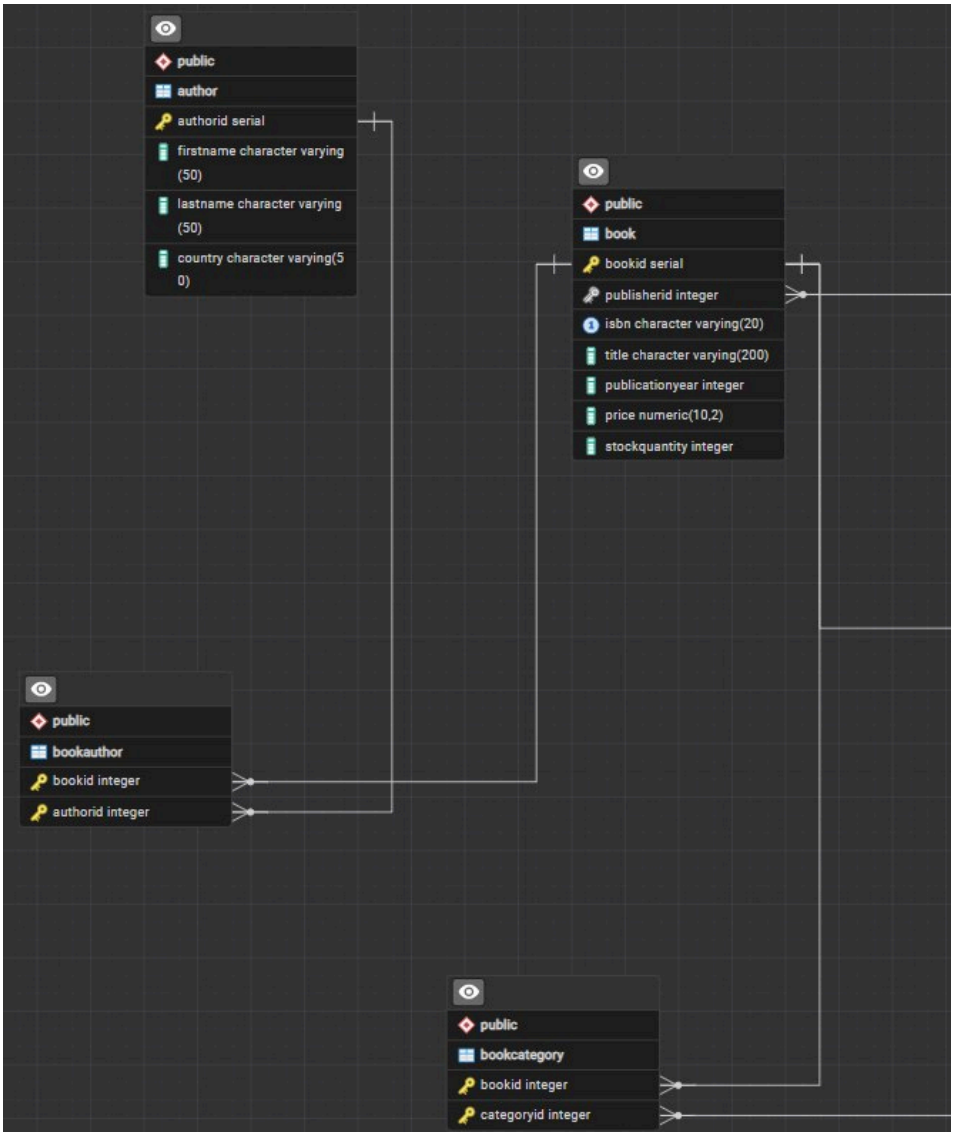
SQL

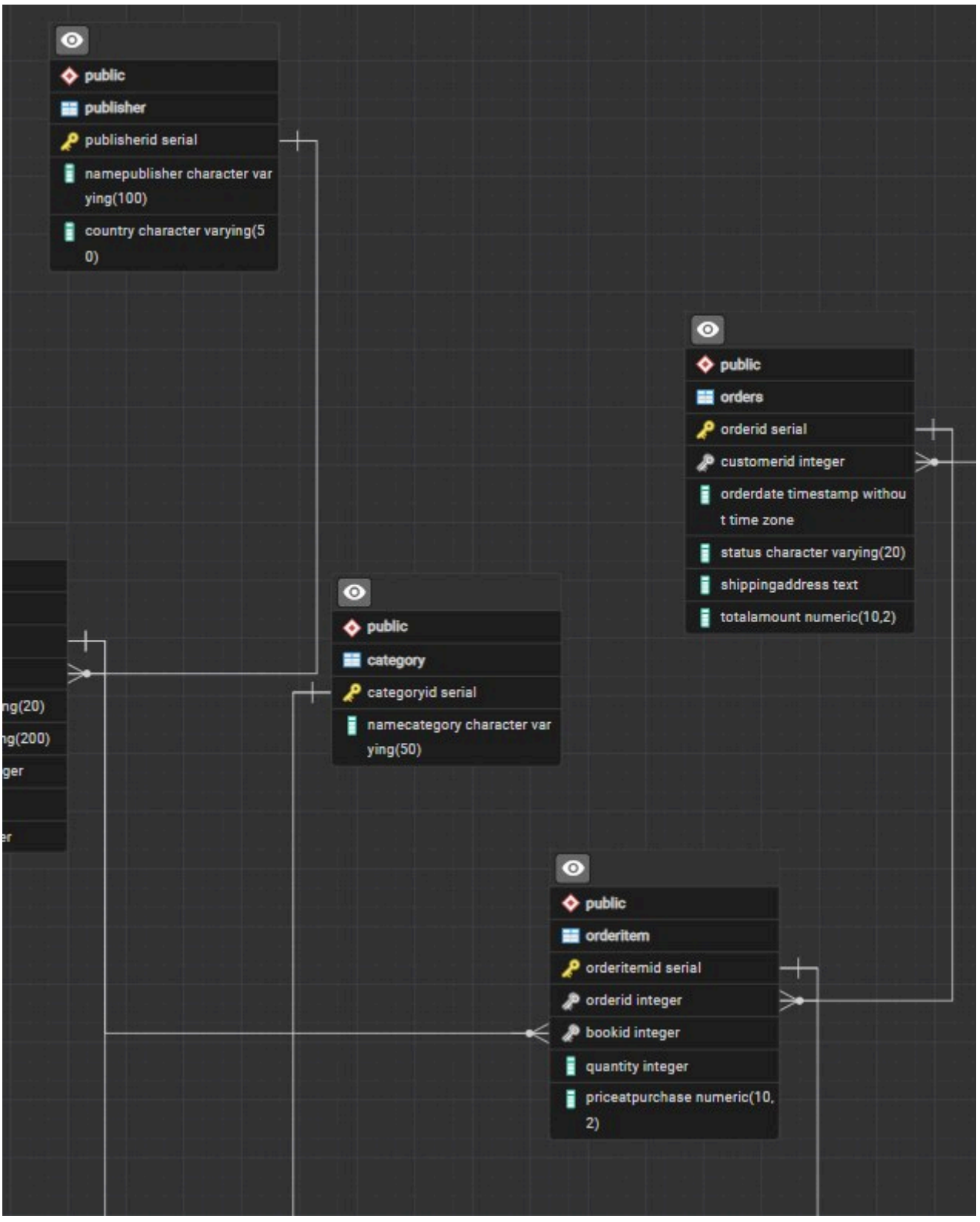
	tbl text	count bigint
1	customer	4
2	publisher	3
3	author	4
4	category	4
5	book	4
6	bookauthor	4
7	bookcatego...	4
8	orders	4
9	orderitem	5
10	review	5

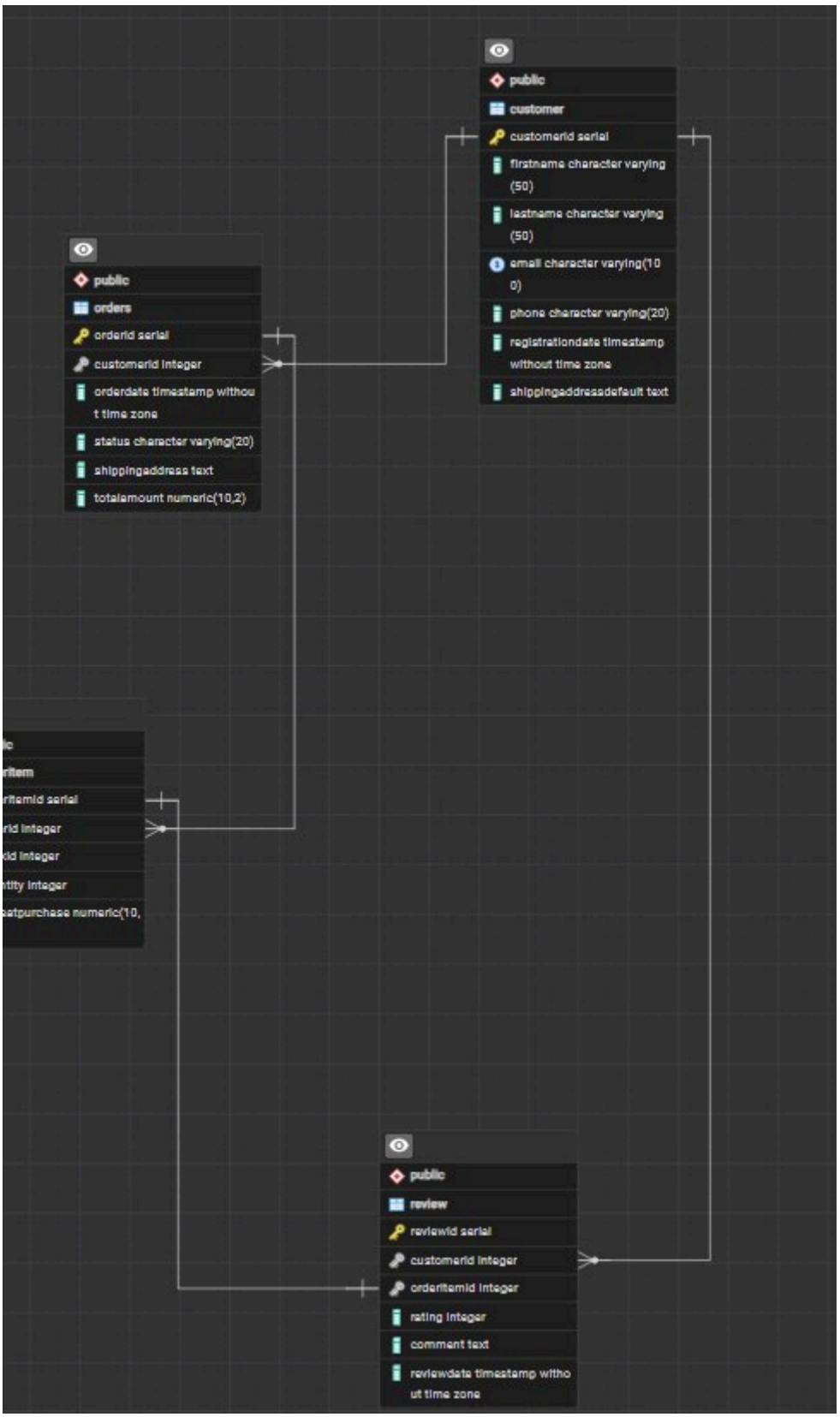
ER-діаграма:

Структуру бази даних було візуалізовано за допомогою вбудованих інструментів СУБД, що підтвердило наявність усіх зв'язків між первинними (PK) та зовнішніми (FK) ключами згідно із заданою архітектурою.


public
publisher
publisherid serial
namepublisher character varying(100)
country character varying(50)
public
author
authorid serial
firstname character varying (50)
lastname character varying (50)
country character varying(50)
public
book
bookid serial
publisherid integer
isbn character varying(20)
title character varying(200)
publicationyear integer
price numeric(10,2)
stockquantity integer
public
category
categoryid serial
namecategory character varying(50)
public
orders
orderid serial
customerid integer
orderdate timestamp without time zone
status character varying(20)
shippingaddress text
totalamount numeric(10,2)
public
customer
customerid serial
firstname character varying (50)
lastname character varying (50)
email character varying(100)
phone character varying(20)
registrationdate timestamp without time zone
shippingaddressdefault text
public
orderitem
orderitemid serial
orderid integer
bookid integer
quantity integer
priceatpurchase numeric(10,2)
public
bookauthor
bookid integer
authorid integer
public
bookcategory
bookid integer
categoryid integer
public
review
reviewid serial
customerid integer
orderitemid integer
rating integer
comment text
reviewdate timestamp without time zone


public
author
authorid serial
firstname character varying (50)
lastname character varying (50)
country character varying(50)
public
book
bookid serial
publisherid integer
isbn character varying(20)
title character varying(200)
publicationyear integer
price numeric(10,2)
stockquantity integer
public
bookauthor
bookid integer
authorid integer
public
bookcategory
bookid integer
categoryid integer


public
publisher
publisherid serial
namepublisher character varying(100)
country character varying(50)
public
orders
orderid serial
customerid integer
orderdate timestamp without time zone
status character varying(20)
shippingaddress text
totalamount numeric(10,2)
public
category
categoryid serial
namecategory character varying(50)
public
orderitem
orderitemid serial
orderid integer
bookid integer
quantity integer
priceatpurchase numeric(10,2)



Висновок:

Під час виконання лабораторної роботи було успішно перетворено концептуальну ER-модель інтернет-магазину книг на фактичну реляційну схему в СУБД PostgreSQL. За допомогою інструкцій DDL створено 10 взаємопов'язаних таблиць із чітким визначенням відповідних типів даних, первинних та зовнішніх ключів, а також обмежень NOT NULL, UNIQUE, CHECK та DEFAULT для забезпечення суворої цілісності даних. Заповнення таблиць тестовими даними за допомогою інструкцій DML, виконання аналітичних JOIN-запитів та успішне проведення

негативних тестів у середовищі pgAdmin підтвердили правильність побудови схеми, коректність роботи всіх зв'язків та захист бази даних від введення некоректних даних. У результаті роботи було здобуто практичні навички проектування, реалізації та адміністрування реляційних баз даних.